

AGOSTO DE 2026

ATELIER ÉLIE

PROYECTO DE PROGRAMACIÓN WEB Y PHP

PEDRO ELÍAS SIMÓN VELÁSQUEZ

1. Descripción General del Proyecto y Objetivos

1.1. Resumen Funcional

He desarrollado **Atelier Élie**, una plataforma de comercio electrónico orientada a la venta de productos de sastrería artesanal. La aplicación ha sido concebida para ofrecer una experiencia de usuario fluida, elegante e intuitiva, asegurando al mismo tiempo la integridad de las transacciones y la protección de la información sensible.

Entre las funcionalidades principales que he implementado en el sistema destacan:

- **Catálogo dinámico y filtrado avanzado:** Navegación por productos con capacidades de búsqueda por texto libre y filtrado por atributos como material, color, marca, talla y rango de precio máximo.
- **Gestión de la bolsa de compras (Carrito):** Sistema de persistencia basado en sesiones que permite agregar, incrementar, decrementar o eliminar artículos calculando dinámicamente la base imponible y el IVA (21%).
- **Panel de usuario y autenticación segura:** Registro de nuevos clientes, inicio de sesión protegido contra ataques de fuerza bruta (mediante bloqueo temporal tras varios intentos fallidos) y gestión de perfil, direcciones y contraseñas.
- **Lista de deseos (Favoritos):** Módulo para que los usuarios autenticados guarden sus productos preferidos mediante interacciones de conmutación (*toggle*).
- **Procesamiento de pedidos (Checkout):** Flujo de tramitación de compras transaccional y atómico en base de datos que asegura el descuento de stock en tiempo real y registra el histórico de pedidos.

1.2. Objetivos del Proyecto

Al abordar este proyecto, me fijé una serie de objetivos fundamentales para consolidar mis competencias en desarrollo web *backend* y seguridad de aplicaciones:

- **Implementar arquitectura modular y limpia en PHP:** Estructurar el código sin depender de *frameworks* pesados, centralizando la lógica compartida (conexión a base de datos, utilidades y seguridad) mediante un sistema de autoloader personalizado.
- **Aplicar directivas estrictas de seguridad web (*Hardening*):** Mitigar las principales vulnerabilidades OWASP mediante la sanitización global de entradas/salidas (anti-XSS), consultas preparadas con PDO (anti-SQLi), tokens CSRF criptográficos, cookies de sesión seguras (HttpOnly, SameSite=Strict) y cabeceras de respuesta HTTP defensivas.
- **Garantizar la consistencia de datos en compras:** Utilizar transacciones atómicas de MySQL (START TRANSACTION, COMMIT, ROLLBACK) y bloqueos de fila (FOR UPDATE) para evitar condiciones de carrera (*race conditions*) o descuadres de inventario.

- **Diseñar una interfaz responsiva y temática:** Construir un entorno visual atractivo utilizando Tailwind CSS con una paleta de colores oscuros (*dark mode*) adaptada a la identidad estética de la tienda.

1.3. Stack Tecnológico

Para la construcción y despliegue del proyecto, he seleccionado las siguientes tecnologías:

- **Lenguaje Backend (PHP 8.2+):** Encargado de la lógica de negocio, procesamiento de formularios, control de sesiones y renderizado dinámico de vistas en el servidor.
- **Base de Datos (MySQL 8.0):** Sistema gestor de base de datos relacional para la persistencia de usuarios, catálogo, pedidos, direcciones y favoritos, utilizando el driver PDO de PHP.
- **Maquetación y Estilos (Tailwind CSS):** *Framework* CSS orientado a clases de utilidad para construir una interfaz limpia, adaptada a dispositivos móviles y escritorios.
- **Entorno de Contenedores (Docker & Docker Compose):** Aislamiento del entorno de desarrollo mediante contenedores para la web y la base de datos, garantizando la portabilidad del proyecto sin dependencias del sistema anfitrión.

2. Arquitectura del Sistema

2.1. Estructura y Modelo de la Base de Datos

Para soportar toda la persistencia de **Atelier Élie**, he diseñado una base de datos relacional en MySQL estructurada para mantener la integridad referencial y optimizar el rendimiento de las consultas. La arquitectura consta de seis entidades principales:

1. **USUARIO:** Almacena la información de los clientes registrados.
 - *Campos:* id_user (PK), nombres, apellidos, dni, correo, telefono, contrasenia (hash BCRYPT/DEFAULT), intentos_fallidos y bloqueado_hasta (para el control de fuerza bruta).
2. **DIRECCION:** Guarda la información logística para el envío de los encargos.
 - *Campos:* id_direccion (PK), id_user (FK referenciando a USUARIO), calle, ciudad, codigo_postal.
3. **CORBATA:** Representa el catálogo de productos disponibles en el atelier.
 - *Campos:* id_corbata (PK), talla, color, material, marca, precio, stock, imagen.
4. **PEDIDO:** Registra la cabecera de las compras confirmadas por los usuarios.
 - *Campos:* id_pedido (PK), id_user (FK), id_direccion (FK), fecha_pedido, total, estado.
5. **DETALLE_PEDIDO (o LINEA_PEDIDO):** Tabla intermedia que resuelve la relación N:M entre pedidos y productos.
 - *Campos:* id_pedido (FK), id_corbata (FK), cantidad, precio_unitario.
6. **FAVORITO:** Tabla intermedia que gestiona la lista de deseos personalizada de cada usuario.
 - *Campos:* id_user (FK), id_corbata (FK).

Relaciones del Modelo Relacional:

- **USUARIO – DIRECCION (1:N):** Un usuario puede registrar múltiples direcciones de entrega, pero cada dirección pertenece a un único usuario.
- **USUARIO – PEDIDO (1:N):** Un usuario puede efectuar múltiples pedidos a lo largo del tiempo, perteneciendo cada pedido a un único cliente.
- **DIRECCION – PEDIDO (1:N):** Una dirección guardada puede utilizarse para recibir múltiples pedidos, pero cada pedido se envía a una dirección concreta.
- **PEDIDO – CORBATA (N:M):** Un pedido puede contener múltiples corbatas y una corbata puede comercializarse en múltiples pedidos; esta relación se materializa mediante la tabla de ruptura DETALLE_PEDIDO.

- **USUARIO – CORBATA vía FAVORITO (N:M):** Un usuario puede marcar múltiples corbatas como favoritas y un producto puede ser guardado por varios usuarios.

2.2. Organización Modular y Arquitectura de Código (helpers/)

Con el objetivo de evitar código duplicado y mantener una separación clara de responsabilidades, he organizado el *backend* de la aplicación bajo un enfoque modular desacoplado. En lugar de incluir librerías de terceros pesadas, he construido un directorio centralizado de utilidades (helpers/) que orquesta la seguridad, la conexión a datos y los servicios transversales:

www/

- └─ assets/ # Recurso estáticos (imágenes de catálogo y modelos)
- └─ helpers/ # Módulo de utilidades y servicios globales
 - └─ autoload.php # Cargador global y configuración de sesión segura
 - └─ db.php # Conexión centralizada a la BD (PDO Singleton)
 - └─ csrf.php # Generación y verificación de tokens anti-CSRF
 - └─ sanitizer.php # Funciones de escape Anti-XSS (e(), sanear_string())
 - └─ headers.php # Inyección de cabeceras HTTP de seguridad (CSP, XFO...)
 - └─ └─ logger.php # Gestor de logs de errores y excepciones
- └─ logs/ # Directorio privado de registros del sistema
- └─ └─ .htaccess # Bloqueo de acceso web público (Require all denied)
- └─ header.php / footer.php # Plantillas de maquetación de la interfaz
- └─ index.php # Portada principal con productos destacados
- └─ catalogo.php # Buscador y filtrado dinámico del catálogo
- └─ producto.php # Vista de detalle e historias de las piezas
- └─ carrito.php # Gestión de bolsa de compras
- └─ checkout.php # Procesamiento atómico de pedidos
- └─ cuenta.php # Panel de control del usuario y favoritos
- └─ login.php / registro.php # Autenticación y gestión de acceso
- └─ └─ logout.php # Cierre y destrucción segura de sesión

Responsabilidades de los componentes en helpers/:

- **autoload.php:** Actúa como el punto de entrada de cada petición. Inicia la sesión aplicando las directivas estrictas de cookies (HttpOnly, SameSite=Strict, Secure) y carga automáticamente el resto de los módulos helpers.
- **db.php:** Centraliza las credenciales de la base de datos MySQL en una única función helper (obtener_conexion_db()), evitando instanciar múltiples conexiones PDO en cada vista.
- **csrf.php:** Genera tokens criptográficos aleatorios (random_bytes(32)) para proteger todos los formularios POST contra ataques de falsificación de peticiones en sitios cruzados.
- **sanitizer.php:** Proporciona funciones helper globales como e() para escapar caracteres especiales HTML mediante htmlspecialchars(), evitando vulnerabilidades de Cross-Site Scripting (XSS).
- **headers.php:** Emite respuestas con cabeceras de seguridad de nivel industrial (X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Content-Security-Policy).
- **logger.php:** Captura excepciones no controladas en el servidor, las registra de forma privada con marca de tiempo en logs/app_errors.log y muestra una interfaz genérica y sobria al cliente final para evitar fugas de información técnica.

3. Implementación de Seguridad (*Hardening*)

Dado que las aplicaciones de comercio electrónico manejan datos personales y transacciones financieras, he diseñado **Atelier Élie** bajo el principio de *seguridad en profundidad* (*Defense in Depth*). He integrado múltiples capas defensivas para mitigar las vulnerabilidades más críticas identificadas en el estándar OWASP Top 10:

3.1. Protección contra Falsificación de Peticiones en Sitios Cruzados (Tokens CSRF)

Para evitar que sitios externos maliciosos fuercen a un usuario autenticado a realizar acciones no deseadas (como tramitar un pedido o alterar su perfil), he implementado un sistema estricto de validación de tokens anti-CSRF:

- **Generación aleatoria criptográfica:** Genero tokens únicos asociados a la sesión mediante `random_bytes(32)`.
- **Verificación de tiempo constante:** Todos los formularios que envían peticiones POST incluyen un campo oculto con este token. Al procesar el formulario, utilizo la función `hash_equals()` para comparar el token recibido con el guardado en la sesión. Esto evita ataques de temporización (*timing attacks*) que intenten deducir el valor del token.

3.2. Sanitización y Prevención de Scripts en Sitios Cruzados (Anti-XSS)

Con el fin de evitar que un atacante inyecte código JavaScript malicioso en los campos de entrada o en el renderizado de las vistas, he desarrollado el módulo `helpers/sanitizer.php`:

- **Escapado de salida (`e()`):** Aplico la función helper `e()` basada en `htmlspecialchars($var, ENT_QUOTES, 'UTF-8')` en todas las impresiones dinámicas de variables en la interfaz HTML. Esto neutraliza caracteres especiales como `<`, `>`, `"` o `'`.
- **Sanitización de entrada (`sanear_string()`):** Higienizo las variables del formulario eliminando espacios innecesarios y filtrando etiquetas indeseadas mediante `trim()` y `strip_tags()` antes de procesarlas.

3.3. Configuración Segura de Cookies de Sesión (Session Cookie Hardening)

La cookie de sesión `PHPSESSID` representa la identidad del usuario autenticado. Para prevenir su robo o manipulación, he configurado sus directivas en `helpers/autoload.php` mediante `session_set_cookie_params()` antes de llamar a `session_start()`:

- **HttpOnly (`true`):** Bloquea completamente el acceso a la cookie mediante scripts de cliente (como `document.cookie`), inutilizando intentos de robo de sesión vía XSS.

- **SameSite=Strict:** Le exige al navegador no enviar la cookie de sesión en peticiones procedentes de orígenes externos, constituyendo una segunda capa defensiva clave contra ataques CSRF.
- **Secure:** Configurado para activarse cuando la conexión se realiza sobre HTTPS, impidiendo que la cookie se transmita en texto plano por canales no cifrados.
- **Prevención de fijación de sesión:** Invoco `session_regenerate_id(true)` inmediatamente tras un inicio de sesión exitoso para renovar el identificador de sesión.

3.4. Cabeceras HTTP de Seguridad (Security Headers)

Mediante el módulo `helpers/headers.php`, fuerzo a la aplicación a emitir respuestas HTTP defensivas en cada petición que realiza el navegador:

- **X-Frame-Options: DENY:** Impide que la tienda sea incrustada en marcos (`<iframe>`) de webs externas, previniendo ataques de *Clickjacking*.
- **X-Content-Type-Options: nosniff:** Desactiva la deducción automática del tipo MIME por parte del navegador, evitando que archivos de datos o imágenes sean ejecutados como código.
- **Content-Security-Policy (CSP):** Define una política de seguridad de contenido estricta que especifica desde qué orígenes permitidos se pueden cargar *scripts*, estilos e imágenes (`default-src 'self'`), restringiendo ejecuciones no autorizadas de fuentes externas.

3.5. Transacciones SQL Atómicas y Control de Concurrencia (FOR UPDATE)

En el flujo de tramitación del pedido (`checkout.php`), es crítico evitar que dos usuarios compren el último producto en stock simultáneamente (*race conditions*). Para solucionarlo, he implementado transacciones PDO aisladas:

- Inicio la transacción explícitamente con `$pdo->beginTransaction()`.
- Utilizo la cláusula **SELECT ... FOR UPDATE** sobre las filas de la tabla CORBATA que están siendo compradas. Esto bloquea temporalmente los registros en la base de datos hasta que la transacción finaliza.
- Verifico que el stock real sea suficiente. Si es correcto, descuento el inventario e inserto los registros en PEDIDO y DETALLE_PEDIDO.
- Confirmo los cambios de forma permanente con `$pdo->commit()`. Si se produce cualquier fallo o falta de stock, ejecuto un `$pdo->rollBack()` para devolver la base de datos a su estado original de forma limpia.

3.6. Manejo Global de Excepciones y Registro de Logs Privado

Para cumplir con los estándares de producción y evitar la fuga de información sensible (como nombres de tablas, credenciales o *stack traces* de errores de MySQL):

- **Ocultación al usuario:** Configuré un manejador global de excepciones con `set_exception_handler()` en `helpers/logger.php`. En caso de fallo crítico en el servidor, el usuario ve una pantalla de error genérica y elegante sin detalles técnicos.
- **Auditoría interna (`app_errors.log`):** El sistema registra automáticamente el mensaje de error completo, la fecha, la hora, la dirección IP del cliente y la URI afectada en un archivo privado `logs/app_errors.log`.
- **Protección del directorio de registros:** La carpeta `logs/` incluye un archivo `.htaccess` con la regla `Require all denied` para impedir el acceso o descarga directa de los archivos de log a través del navegador.